Abstract Factory

Abstract Factory · Light/Dark UI widget family stays consistent · Build with Waqar

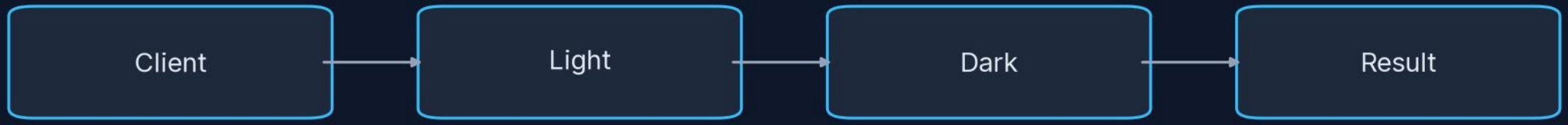
Today's Hook

- Problem: Light/Dark UI widget family stays consistent
- Pattern: Abstract Factory — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Light/Dark UI widget family stays consistent
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Abstract Factory: Light/Dark UI widget family stays consistent

Real-World Scenario

- Running example: Light/Dark UI widget family stays consistent
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

public final class NaiveUi {
  public static String render(String theme) {
    if ("light".equals(theme)) return "light-button+dark-checkbox"; // BUG: mixed family
    return "dark-button+light-checkbox";
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

interface Button { String paint(); }
interface Checkbox { String paint(); }
interface UiFactory { Button button(); Checkbox checkbox(); }
record LightButton() implements Button { public String paint(){return "light-button";} }
record DarkButton() implements Button { public String paint(){return "dark-button";} }
record LightCheckbox() implements Checkbox { public String paint(){return "light-checkbox";} }
record DarkCheckbox() implements Checkbox { public String paint(){return "dark-checkbox";} }
record LightFactory() implements UiFactory {
  public Button button(){return new LightButton();} public Checkbox checkbox(){return new LightCheckbox();}
}
record DarkFactory() implements UiFactory {
  public Button button(){return new DarkButton();} public Checkbox checkbox(){return new DarkCheckbox();}
}
final class Dialog {
  private final UiFactory f; public Dialog(UiFactory f){this.f=f;}
  public String render(){ return f.button().paint()+"+"+f.checkbox().paint(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Abstract Factory: Light/Dark UI family ===");
    System.out.println("--- naive (mixed family bug) ---");
System.out.println(NaiveUi.render("light"));
System.out.println("--- abstract factory ---");
System.out.println(new Dialog(new LightFactory()).render());
System.out.println(new Dialog(new DarkFactory()).render());
  }
}
```

Walkthrough Output

```
=== Abstract Factory: Light/Dark UI family ===
--- naive (mixed family bug) ---
light-button+dark-checkbox
--- abstract factory ---
light-button+light-checkbox
dark-button+dark-checkbox
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Abstract Factory.

Related Patterns

- Contrast Abstract Factory with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Abstract Factory → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.